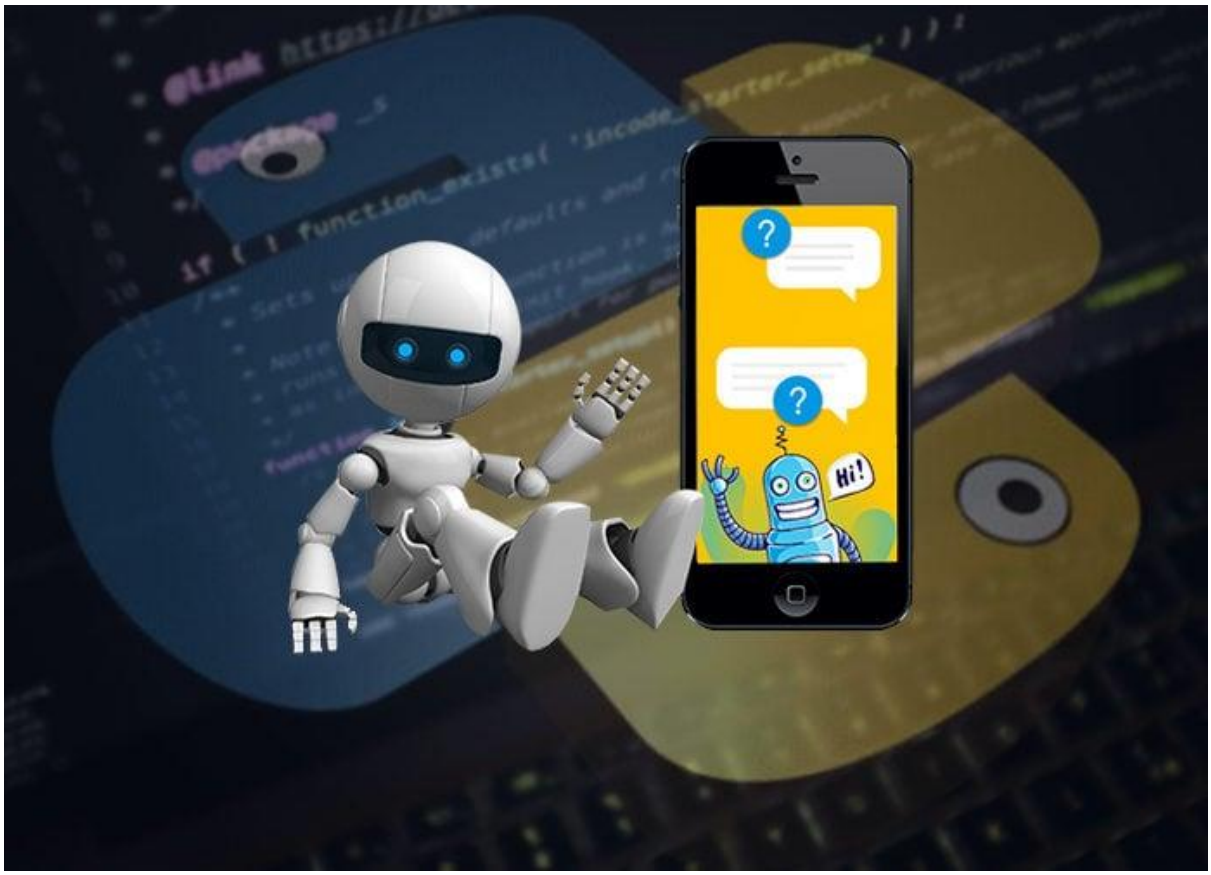
AIML Capstone Project – NLP1 Group 1 Final Report

Industrial Safety NLP based Chatbot



Project Final Report prepared by

Amrutha Ananthakrishna

Sai Aditya Vinjanampati

Ninad Mahalle

Hemanth Pv

Loka Sai Eswar Vuyyuru

Presented to Great learning Mentor Mr. Aniket Chhabra

Date: 14-Jan-2023

Table of Contents

Problem Statement..... 3

Step-1: Importing Libraries, Data and Info of Data 4

Step 2: Data Cleansing……..5

Step 3: Data Preprocessing (NLP Preprocessing Techniques) ……..7

Step 4: Data preparation - Cleansed data in .xlsx or .csv file……19

Step 5: Design train and test basic machine learning classifiers…….19

Observations from Milestone-1…….21

Milestone-2**:**

Step 1: Design, train and test Neural networks classifiers……..……22

Step 2: Design, train and test RNN or LSTM classifiers**……..**……28

Step 3: Choose the best performing classifier and pickle it……31

Final Observations…….34

Problem Statement

• **DOMAIN**: Industrial safety. NLP based Chatbot.

• **CONTEXT**: The database comes from one of the biggest industry in Brazil and in the world. It is an urgent need for industries/companies around the globe to understand why employees still suffer some injuries/accidents in plants. Sometimes they also die in such environment.

• **DATA DESCRIPTION**:

The database is basically records of accidents from 12 different plants in 03 different countries which every line in the data is an occurrence of an accident.

Columns description:

▸ **Data**: timestamp or time/date information

▸ **Countries**: which country the accident occurred (anonymised)

▸ **Local**: the city where the manufacturing plant is located (anonymised)

▸ **Industry sector**: which sector the plant belongs to

▸ **Accident level**: from I to VI, it registers how severe was the accident (I means not severe but VI means very severe)

▸ **Potential Accident Level**: Depending on the Accident Level, the database also registers how severe the accident could have been (due to other factors involved in the accident)

▸ **Genre**: if the person is male of female

▸ **Employee or Third Party**: if the injured person is an employee or a third party

▸ **Critical Risk**: some description of the risk involved in the accident

▸ **Description**: Detailed description of how the accident happened.

Link to download the dataset: https://www.kaggle.com/ihmstefanini/industrial-safety-and-health-analytics-database [for your reference only]

• **PROJECT OBJECTIVE**: Design a ML/DL based chatbot utility which can help the professionals to highlight the safety risk as per the incident description

Step-1: Importing Libraries, Data and Info of Data

In this section we have imported the necessary libraries and loaded the data and obtained the data information.

Shape of the Data Frame:

The data frame consists of 425 rows and 11 columns

```
#   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Unnamed: 0                425 non-null    int64
 1   Data                      425 non-null    object
 2   Countries                 425 non-null    object
 3   Local                     425 non-null    object
 4   Industry Sector           425 non-null    object
 5   Accident Level            425 non-null    object
 6   Potential Accident Level  425 non-null    object
 7   Genre                     425 non-null    object
 8   Employee or Third Party   425 non-null    object
 9   Critical Risk             425 non-null    object
 10  Description               425 non-null    object
dtypes: int64(1), object(10)
memory usage: 36.6+ KB
```

The data are of Object type and there seemed to be and unnamed column which is not required and can be dropped. Some of the columns needs to be renamed to match the description and data variable seems to be object but it is a date column. We will perform all these activities in Data cleansing.

Step 2: Data Cleansing

- Data we receive or collect usually isn't in the form that can be fed directly to the Machine Learning algorithm.
- More often than not, data has anomalies that need to be cleaned
- Data might sometimes contain duplicate entries of the same record. Such instances do not contribute towards modelling and can be removed
- Data might also contain columns that have a high cardinality or columns that are created when loading data

Dropping and Renaming Columns:

We have started by dropping the unnamed column as it doesn't contribute to our analysis and renamed the variables.

- Data : Date
- Countries : Country
- Genre : Gender
- Employee or Third Party : type

Removing Duplicates :

We have found 7 duplicates after dropping and renaming the columns.

	Date	Country	Local	Industry Sector	Accident Level	Potential Accident Level	Gender	type	Critical Risk	Description
77	2016-04-01 00:00:00	Country_01	Local_01	Mining	I	V	Male	Third Party (Remote)	Others	In circumstances that two workers of the Abrat...
262	2016-12-01 00:00:00	Country_01	Local_03	Mining	I	IV	Male	Employee	Others	During the activity of chuteo of ore in hopper...
303	2017-01-21 00:00:00	Country_02	Local_02	Mining	I	I	Male	Third Party (Remote)	Others	Employees engaged in the removal of material f...
345	2017-03-02 00:00:00	Country_03	Local_10	Others	I	I	Male	Third Party	Venomous Animals	On 02/03/17 during the soil sampling in the re...
346	2017-03-02 00:00:00	Country_03	Local_10	Others	I	I	Male	Third Party	Venomous Animals	On 02/03/17 during the soil sampling in the re...
355	2017-03-15 00:00:00	Country_03	Local_10	Others	I	I	Male	Third Party	Venomous Animals	Team of the VMS Project performed soil collect...
397	2017-05-23 00:00:00	Country_01	Local_04	Mining	I	IV	Male	Third Party	Projection of fragments	In moments when the 02 collaborators carried o...

After dropping those duplicates the new shape of our data is (418,10).

Checking Unique values and frequencies:

Below are our observations.

- Potential Accident Level: There's only one incident labelled as "VI," which might be considered an outlier based on the rarity of this level. Gender: The gender

distribution seems unexpected based on the nature of the incidents and the industry. The industry might be predominantly male dominated.

- There are only two Gender types.
- There are three Country types, Industry sector types, Employee types.
- There are five Accident Level types.
- There are six Potential Accident Level types.
- There are twelve Local cities where manufacturing plant is located.
- Above mentioned data column values are in sequence so there are no outliers in these columns.
- There are records of accidents from 1st Jan 2016 to 9th July 2017 in every month. There are no outliers in date column.
- There are lot of Critical risk descriptions. But we didn’t see any outliers.

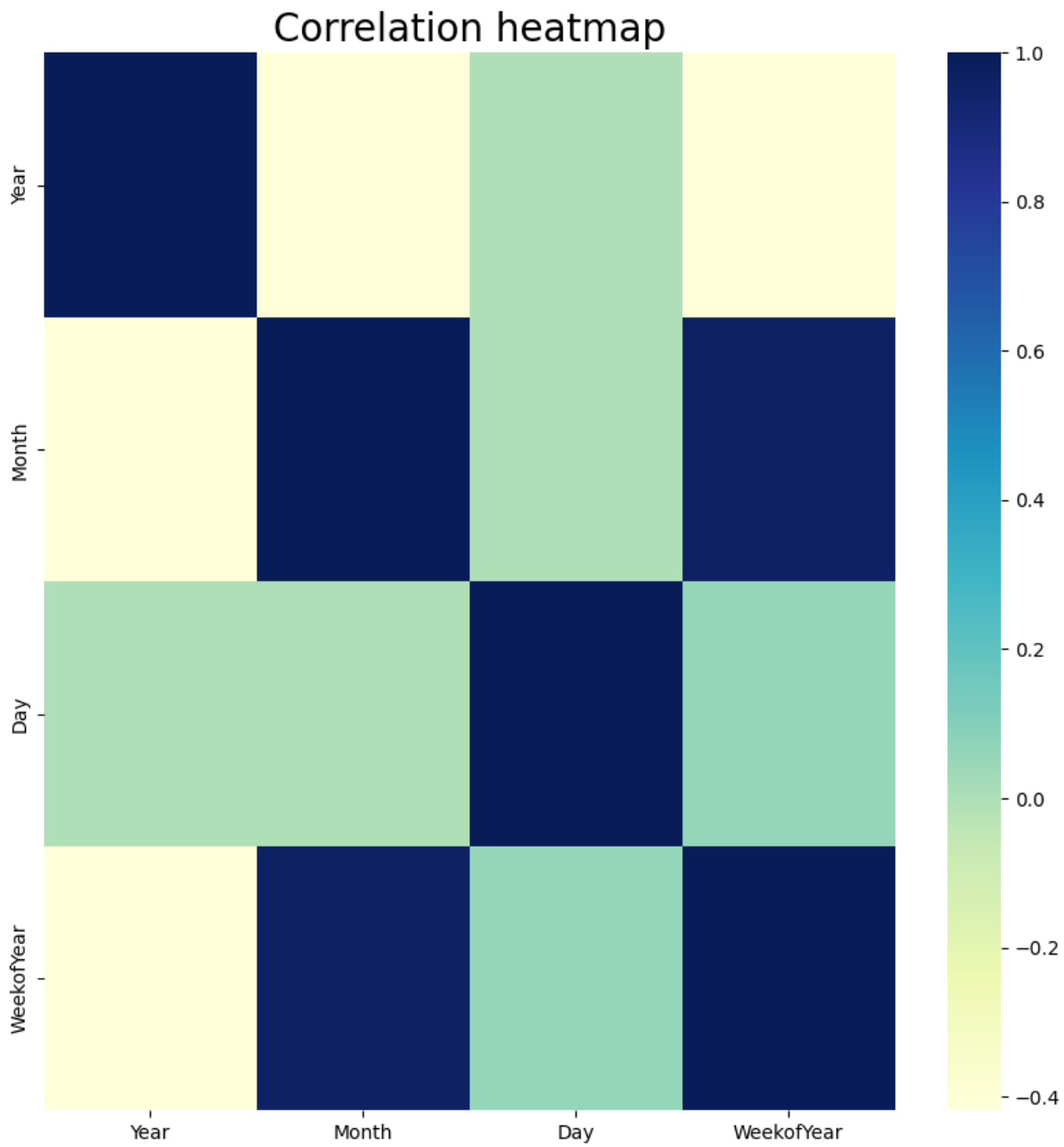
Activities performed in Data Cleansing:

1. Removed unwanted column (Unnamed: 0).
2. Renamed few columns ('Data', 'Countries', 'Genre', 'Employee or Third Party')
3. Found seven duplicate records in the data. Dropped them.
4. There are no outliers in the dataset.
5. There are no missing values in dataset.
6. After data cleansing,
7. We have 418 rows and 10 columns.

Step 3: Data Preprocessing (NLP Preprocessing Techniques)

In this step we have performed correlation and Univariate analysis.

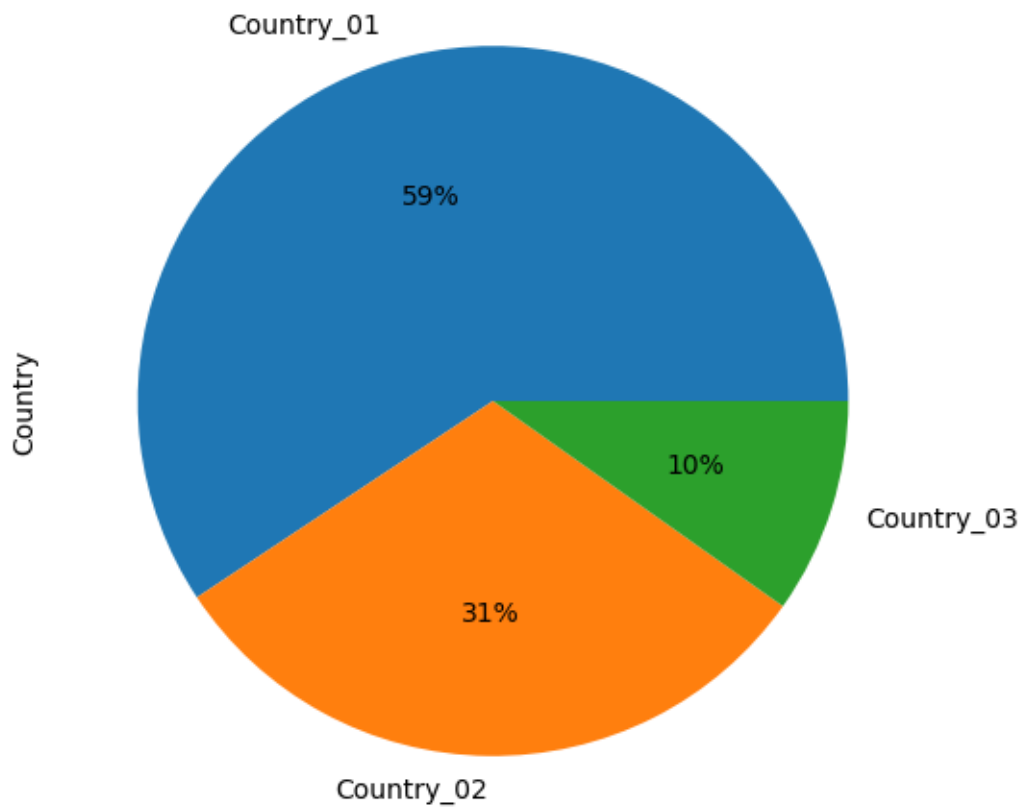
Correlation Heatmap of data:



Univariate Analysis:

Univariate Analysis of Country:

```
Value Counts for `Country` label
------------------------------------------------------------
Country_01 count: 248 i.e. 59.0%
Country_02 count: 129 i.e. 31.0%
Country_03 count: 41 i.e. 10.0%
------------------------------------------------------------
Distributon of `Country` label
------------------------------------------------------------
```



Percentage of accidents occurred in Country_01 is 59% and 31% in Country_02 and 10% in Country_03.

Country_01 has higher percentage of accidents (more than 50%).

Univariate analysis of accidents occurred in Particular seasons :

```
Incidents occoured in each seasons
------------------------------------------------------------
summer_count: 123
spring_count: 58
autumn_count: 141
winter_count: 96
```

There are less incidents occurred in spring season and more incidents occurred in Autumn season. Even in summer and winter seasons there are good amount of incidents.

Univariate analysis of Gender:

```
Incidents gender wise
------------------------------------------------------------
male_count: 396
female_count: 22
------------------------------------------------------------
Percentage of Incidents gender wise
------------------------------------------------------------
male_count: 95.0%
female_count: 5.0%
```

Proportion of accident levels in each gender is not equal.Males have a very higher accident levels than females.

Univariate analysis of accidents occurred in Local:

Local_03 location has the **highest number** of plants, while Local_09 and Local_11 have lowest.

Univariate analysis of accidents occurred in individual Industry Sector:

```
Value Counts for `Industry Sector` label
-------------------------------------------------------------
Mining count: 237 i.e. 57.0%
Metals count: 134 i.e. 32.0%
Others count: 47 i.e. 11.0%
```

57% manufacturing plants belongs to Mining sector.

32% manufacturing plants belongs to Metals sector.

11% manufacturing plants belongs to Other sector.

Univariate analysis of Accident Level and Potential Accident Level:

```
Value Counts for `Accident Level` label
----------------------------------------------------------------------
---------
Accident Level - I count: 309 i.e. 74.0%
Accident Level - II count: 40 i.e. 10.0%
Accident Level - III count: 31 i.e. 7.0%
Accident Level - IV count: 30 i.e. 7.0%
Accident Level - V count: 8 i.e. 2.0%
Accident Level - VI count: 0 i.e. 0.0%
-------------------------------------------------------------
Value Counts for `Potential Accident Level
----------------------------------------------------------------------
---------
Potential Accident Level - I count: 45 i.e. 11.0%
Potential Accident Level - II count: 95 i.e. 23.0%
Potential Accident Level - III count: 106 i.e. 25.0%
Potential Accident Level - IV count: 141 i.e. 34.0%
Potential Accident Level - V count: 30 i.e. 7.0%
Potential Accident Level - VI count: 1 i.e. 0.0%
```

- For all levels, the number of accidents decreases as the Accident Level increases.
- For first four levels, the number of accidents increases as the Potential Accident Level increases.

Univariate analysis of Employee type:

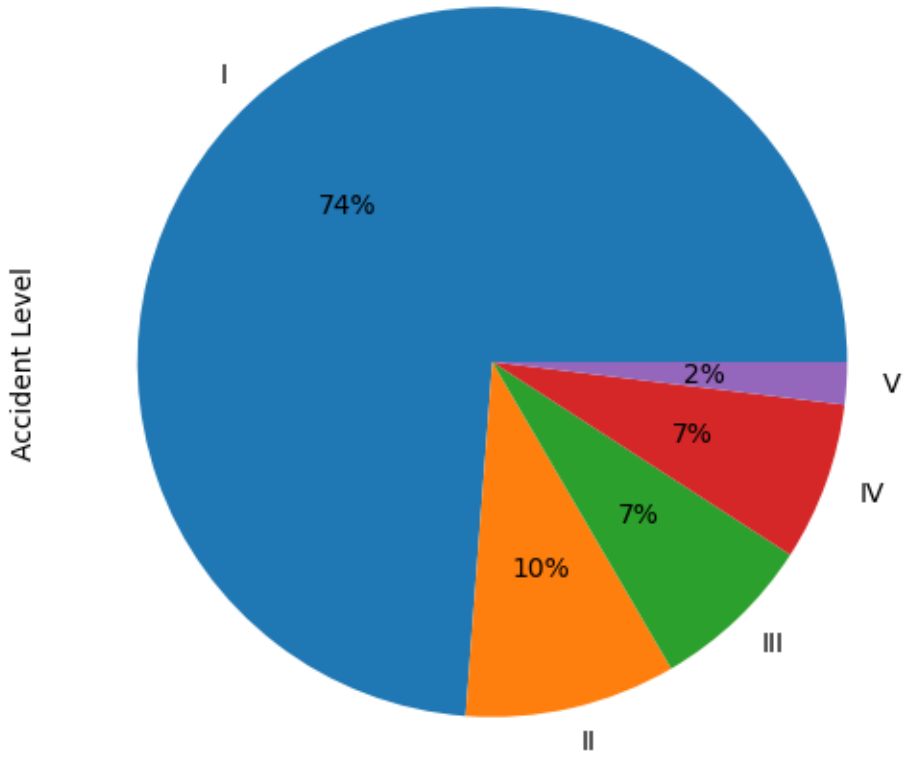
```
Value Counts for `Employee type` label
-------------------------------------------------------------
Third Party count: 185 i.e. 44.0%
Employee count: 178 i.e. 43.0%
Third Party (Remote) count: 55 i.e. 13.0%
```

- 43% own employees working in this industry.
- 44% Third party employees working in this industry.
- 13% Third party (Remote) employees working in this industry.

5 random Descriptions and accident levels from the data where the length of headline is > 100:

```
5 random Descriptions and accident_levels from the data where the length of
Description is > 100
-------------------------------------------------------------------------------
-----------------------------------
Description: Employee was doing cleaning near the area of the pneumatic con
veyor when he was hit by a drop of sulfuric acid on his upper lip.
accident_level: I

Description: The employee of partner company GR reports that she was cuttin
g watermelon when she injured her 2nd chirodactilo of the left hand with th
e knife.
accident_level: II

Description: In the Raise chamber, when the operator and assistant removed
the drilling bar from the 71R drilling machine - weight approx. 500 Kg - ho
lding it at the top with a 2 TM stroke and assisted with jibs to move horiz
ontally, the operator's jib on the bottom slides by moving the top of the d
rill bar causing the assistant's bar to enforce his assistant's hand betwee
n the bar and the piston of the equipment.
accident_level: I

Description: The mincing team was carrying out activities in the city of Ju
ína and was coordinated by mining technician Felipe a time when the mining
technician was last in line and more away from the team, was bitten by a bl
ackjack on the left side of his face. There was no allergic manifestation a
nd the team continued the work. In the afternoon, after lunch, the employee
sought medical care, was medicated and released to continue activities the
next day.
accident_level: I

Description: When checking voltage at the power outlet of the the plug and
socket to make sure the connection was correct, when there was a small elec
trical arc in the power cord, causing a slight burn in the right hand wrist
. The protection system acted immediately.
accident_level: I
```

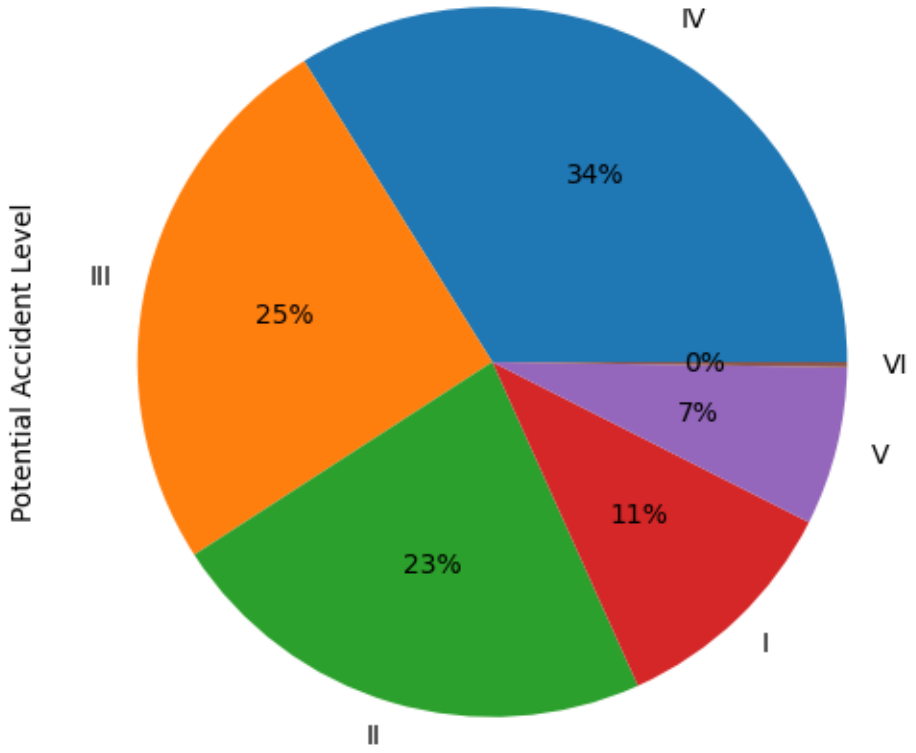
```
--------------------------------------------------------------------------------
Distributon of accident_level where the length of Description is > 100
--------------------------------------------------------------------------------
```



Checking 5 random Descriptions and Potential accident levels from the data where the length of headline is > 100:

```
--------------------------------------------------------------------------------
Checking 5 random Descriptions and pot_accident_levels from the data where
the length of Description is > 100
--------------------------------------------------------------------------------
Description: In moments that the operator of the Jumbo 2, tried energize yo
ur equipment to proceed to the installation of 4 split set at intersection
544 of Nv 3300, remove the lock and opening the electric board of 440V and
400A, and when lifting the thermomagnetic key This makes phase to ground -
phase contact with the panel shell - producing a flash which reaches the op
erator causing the injury described.
pot_accident_level: V

Description: In circumstances that the collaborator performed the cleaning
of the ditch 3570, 0.50 cm deep, removing the pipe of 2 "HDPE material with
an estimated weight of 30 Kg. Together with two collaborators, when pushing
the tube to drain the dune, the collaborator is hit on the lower right side
lip producing a slight blow to the lip. At the time of the event, the colla
borator had a safety helmet, glasses and gloves.
pot_accident_level: II

Description: The employee was performing maintenance of the blower, when th
e residual water projection occurred on his face. In an attempt to remove P
PE, the injured person has had contact with contaminated gloves with the ri
ght eye causing irritation.
pot_accident_level: II

Description: Being 11:30 am. approximately, in the Nv. 1940, CX-791. Mr. Fr
ancisco - Operator of the Scoop 18, observes a polyethylene pipe thrown on
the road, proceeding to lower his equipment and when was found by lifting t
he pipe to the hook of the right gable, a piece of rock that was suspended
in the lectro-welded mesh, impacting on the operator's right eyebrow, causi
ng the injury.
pot_accident_level: III

Description: At a time when the heavy equipment operator got on the equipme
nt to check the right front headlight (defective) of the scooptram # 181, p
osition on the rim position # 2 his left leg which slides, causing the heav
y equipment operator to fall from a height of 0.60 cm
pot_accident_level: IV
```

```
--------------------------------------------------------------------------------
Distributon of pot_accident_level where the length of Description is > 100
--------------------------------------------------------------------------------
```



Univariate Analysis for Critical Risk:

Most of the Critical Risks are classified as 'Others'. Hence it will take so much time to analyze risks and reasons why the accidents occur.

Univariate Analysis of Year and Month:

Accidents are recorded from 1st Jan 2016 to 9th July 2017 in every month.

There are high number of accidents in 2016 and less in 2017.

Number of Accidents are high in the beginning of year and gradually decreasing later.

Bi-variate Analysis of Gender and Industry Sector:

Proportion of any Industry Sector employees in each gender are not equal.

There are more Female employees in Metals and more Male employees in Mining sector.

Data Pre – Processing: Description Column

- Data might not always be in the form that can readily be consumed by the model
- ML/DL models can't process textual data themselves
- Hence, we have to transform such data i.e., textual data, ourselves before feeding the data to the algorithm
- Natural Language Processing (NLP) is a group of tools and techniques that help us clean, process, and transform textual data to a form that the models can readily consume.

Approach:

1. The column that gives us textual context towards modelling is the Description column. We're applying text-processing techniques to it
2. Whenever dealing with textual data, it's always preferred to make the text case-insensitive.
3. We're converting all the text in the description column to lower case

```
#converting description to lower case
df['Description'] = df['Description'].str.lower()
```

4. NLP techniques try to find the frequencies of occurrence of words in the data and the frequency of occurrence of these words wrt each other
5. We're removing special characters from the description as they mostly don't represent any tangible pattern in the data and are mostly there for punctuation.

```
#Removing Special Characters:
df['Description'] = df['Description'].apply(lambda x: re.sub(r'[^a-zA-Z0-9\s]', '', x))
```

6. The next step is to remove stopwords
7. Stopwords in text processing are those words that occur too frequently in language.
8. Words that typically are stopwords include conjunctions, prepositions, interjections, etc.
9. Stopwords are part of human language/speech for the grammatical correctness of the text and might not always contribute towards the pattern in data while processing text.

```
#Stop words
df['Description'] = df['Description'].apply(lambda x: ' '.join([word for word in word_tokenize(x) if word.lower() not in stop_words]))
```

10. Tokenization in text processing is the process in which we break down the data into smaller chunks called tokens.

11. A token is a small manageable chunk of textual data that can be a word, phrase, or sentence.
12. Tokens typically are words as they are the smallest indivisible and meaningful construct in human language.

```
#Tokenize
#Created new column for Tokenized text
df['tokenized_text'] = df['Description'].apply(word_tokenize)
print(df['tokenized_text'])
```

```
0      [removing, drill, rod, jumbo, 08, maintenance,...
1      [activation, sodium, sulphide, pump, piping, u...
2      [substation, milpo, located, level, 170, colla...
3      [945, approximately, nv, 1880, cx695, ob7, per...
4      [approximately, 1145, circumstance, mechanic, ...
                                ...
420    [approximately, 500, approximately, lifting, k...
421    [collaborator, moved, infrastructure, office, ...
422    [environmental, monitoring, activity, area, em...
423    [employee, performed, activity, stripping, cat...
424    [1000, assistant, cleaned, floor, module, e, c...
Name: tokenized_text, Length: 418, dtype: object
```

13. The next step we perform is called text normalization.
14. Text normalization is when we normalize the data by arriving at the root word of the token
15. The two most popular text normalization techniques are Stemming and Lemmatisation
16. Stemming is the process in which we pass a token to the stemming algorithm that gives us the root word based on a set of preset rules.
17. An example of such a rule could be to remove suffixes such as -s, -es, -iess, -ss, -ing, -ed etc., from the end of a word to arrive at the root word
18. Stemming doesn't always arrive at meaningful root words
19. An example could be when we try to find the stem of the word wolves, it gives us wolv by removing the suffix -es. Wolv isn't a valid root word but could still be used for our analysis.

```python
# Initialize the Porter Stemmer
stemmer = PorterStemmer()

# Define a function to perform stemming on the 'text' column
def stem_words(words):
    return [stemmer.stem(word) for word in words]

# Define a function to perform stemming on the 'text' column
def stem_words(words):
    return [stemmer.stem(word) for word in words]

# Apply the function to the 'text' column and create a new column 'stemmed_text'
df['stemmed_messages'] = df['Description'].apply(stem_words)
```

20. Lemmatization on the other hand is a text normalisation technique that arrives at the root word while preserving the spelling and also making sure the root word is a part of a valid lexicon
21. Lemmatization is generally slower to implement than stemming as it goes through the additional process of making sure the root words are valid
22. We performed both stemming and lemmatization on our token to arrive at the root words.

```python
# initialize lemmatizer
lemmatizer = WordNetLemmatizer()

# define function to lemmatize tokens
def lemmatize_tokens(tokens):
    # convert POS tag to WordNet format
    def get_wordnet_pos(word):
        tag = nltk.pos_tag([word])[0][1][0].upper()
        tag_dict = {"J": wordnet.ADJ,
                    "N": wordnet.NOUN,
                    "V": wordnet.VERB,
                    "R": wordnet.ADV}
        return tag_dict.get(tag, wordnet.NOUN)

    # lemmatize tokens
    lemmas = [lemmatizer.lemmatize(token, get_wordnet_pos(token)) for token in tokens]

    # return lemmatized tokens as a list
    return lemmas

# apply lemmatization function to column of dataframe
df['lemmatized_messages'] = df['Description'].apply(lemmatize_tokens)
```

23. As for working with the other variables, since all of them are categorical too, we can transform them to arrive a numerical features
24. Applied the get_dummies() method on all the categorical columns

```
#Dummy variables encoding
Country_dummies = pd.get_dummies(df['Country'], columns=["Country"], drop_first=True)
Local_dummies = pd.get_dummies(df['Local'], columns=["Local"], drop_first=True)
Gender_dummies = pd.get_dummies(df['Gender'], columns=["Gender"], drop_first=True)
IS_dummies = pd.get_dummies(df['Industry Sector'], columns=['Industry Sector'], prefix='IS', drop_first=True)
EmpType_dummies = pd.get_dummies(df['type'], columns=['type'], prefix='EmpType', drop_first=True)
CR_dummies = pd.get_dummies(df['Critical Risk'], columns=['Critical Risk'], prefix='CR', drop_first=True)
```

25. The get_dummies() method is an implementation of the One-Hot Encoding approach where the unique values in a categorical column are transformed as columns onto themselves and those columns are flagged as 1 for those rows where the relevant label is present
26. We then combine all the above dataframes to a single dataframe
27. Applying Label Encoding to the Season and the Weekday columns

```
# Label encoding
df['Season'] = df['Season'].replace('Summer', 'aSummer').replace('Autumn', 'bAutumn').replace('Winter', 'cWinter').replace('Spring', 'dSpring')
df['Season'] = LabelEncoder().fit_transform(df['Season']).astype(np.int8)

df['Weekday'] = df['Weekday'].replace('Monday', 'aMonday').replace('Tuesday', 'bTuesday').replace('Wednesday', 'cWednesday').replace('Thursday', 'dThursday').replace('Friday', 'eFri
df['Weekday'] = LabelEncoder().fit_transform(df['Weekday']).astype(np.int8)
```

Summary of Data Pre-Processing:

- For length of accident description greater than 100 , 74% of data is captured in low accident level.
- For length of accident description greater than 100 , 34% of data is captured in high medium potential accident level.
- For length of accident description greater than 100 , 25% of data is captured in medium potential accident level.
- For length of accident description greater than 100 , 23% of data is captured in low potential accident level.
- Below are the NLP Pre - processing applied on Accident Description : Converting to lower case. Removing special characters. Removing stop words. Lemmatization.
- After pre-processing steps, Description column has below min and max line length and number of words:
- Minimum line length: 61
- Maximum line length: 672
- Minimum number of words: 9
- Maximum number of words: 98

Step 4: Data preparation - Cleansed data in .xlsx or .csv file

In this step we have performed the below activities

- Remove the columns that are not required for modelling
- Check for Null values
- Check for duplicate records and unique values
- Copy the data frame to CSV and XLSX

Step 5: Design train and test basic machine learning classifiers:

In this step we have created functions to build various models and compared the accuracy results of both training and testing data.

Accuracy Comparison of Models:

	Method	Accuracy of Test data	Accuracy of Train data
0	Logistic Regression	0.440476	1.000000
0	Decision Tree	0.345238	0.398204
0	Random Forest	0.440476	1.000000
0	Adaboost	0.345238	0.431138
0	Bagging (Decision Tree)	0.476190	0.994012
0	Bagging (NB Classifier)	0.416667	0.910180
0	Bagging (LogReg Classifier)	0.416667	1.000000
0	Gradient Boost	0.345238	1.000000
0	SVM	0.321429	1.000000

All performance metrics comparison of each ML models:

	Method	Accuracy of Test data	Precision of Test data	Recall of Test data	F1-score of Test data	Accuracy of Train data	Precision of Train data	Recall of Train data	F1-score of Train data
0	Logistic Regression	0.440476	0.49	0.44	0.40	1.000000	1.00	1.00	1.00
1	Decision Tree	0.345238	0.21	0.35	0.20	0.398204	0.23	0.40	0.26
2	Random Forest	0.440476	0.47	0.44	0.39	1.000000	1.00	1.00	1.00
3	Adaboost	0.345238	0.35	0.35	0.23	0.431138	0.47	0.43	0.33
4	Bagging (Decision Tree)	0.476190	0.53	0.48	0.44	0.994012	0.99	0.99	0.99
5	Bagging (NB Classifier)	0.416667	0.41	0.42	0.36	0.910180	0.93	0.91	0.91
6	Bagging (LogReg Classifier)	0.416667	0.47	0.42	0.34	1.000000	1.00	1.00	1.00
7	Gradient Boost	0.345238	0.40	0.35	0.30	1.000000	1.00	1.00	1.00
8	SVM	0.321429	0.42	0.32	0.17	1.000000	1.00	1.00	1.00

Observations from Milestone-1 :

- We have first imported data. Data cleansing and Data pre-processing is done on the imported data. Exported cleansed data on to the csv and xlsx.
- Function is created to build various models. Nine models have been built using this workflow. They are Logistic Regression , Decision Tree, Random Forest , Adaboost , Bagging using decision tree, Naive Bayes Classifier , Logistic Regression classifier and Gradient Boost, SVM.
- Accuracy results of both training and test data is calculated and added on to the single data frame by using functions.
- When we compare results below are the observations:
- Training data overfitting in most of the models.
- By comparing the results from all above methods, both testing and training data accuracy almost matching in Decision Tree and AdaBoost.
- Lets now compare Decision Tree and AdaBoost:
- Test Accuracy is same in both the models.
- Train Accuracy is slightly high in AdaBoost.
- Precision , Recall and F1-score values of train data is high in Adaboost.
- Precision and F1-score is higher for test data in AdaBoost as compared to Decision tree.
- Recall value of test data is same in both the models.
- Since the data is imbalanced , test data accuracy is same for both the models. So we will compare the performance mertrics of each class.
- For class I both recall and f1 score is higher in Decision Tree but both models have same precision.
- For class II both the models have 0% performance metrics values.
- For class III , all the performance metrics is 0% in Decision Tree but we have 33% precision , 4% recall and 7% F1-score in AdaBoost.
- For class IV , recall and precision values are higher in Decision tree but f1-score is higher in AdaBoost.
- For class V , all the performance metrics is 0% in Decision Tree but we have 50% precision , 20% recall and 29% F1-score in AdaBoost.
- On comparing all performance metrics, we can select the best model as AdaBoost classifier.
- Performance metrics can be improved by using PCA or increase dimensions or by using hyper parameter tuning more efficiently.

Milestone-2: Input: Pre-processed output from Milestone-1

Step 1: Design, train and test Neural networks classifiers:

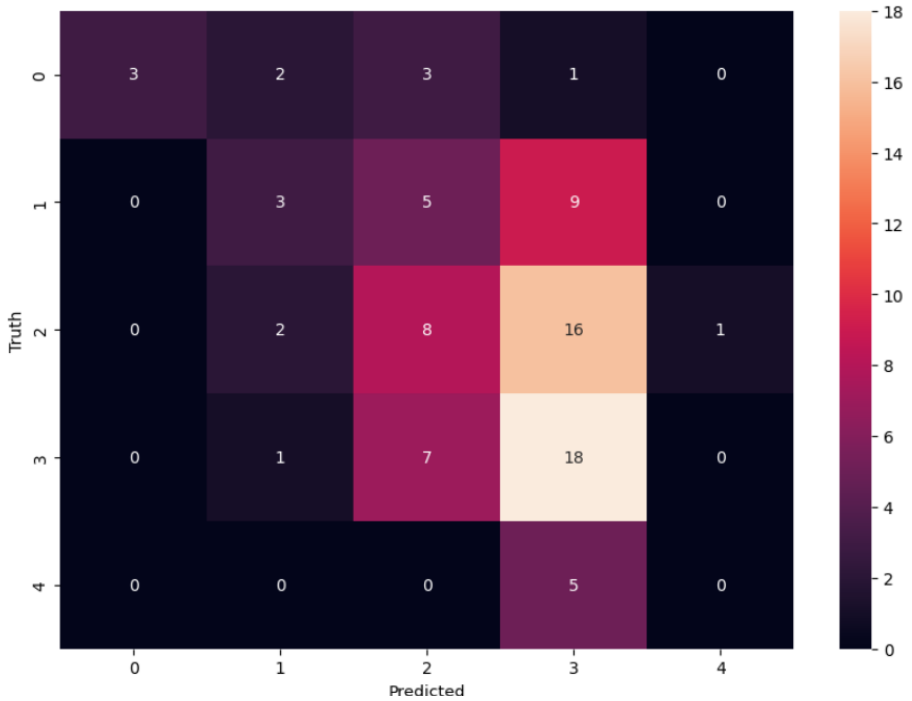
Here we will build two Neural network models (one simple Neural network and one tuned neural network) and then compare the results. We have built the neural networks and attained the below results.

Simple Neural Networks:

```
#Classification Report for test data
print(classification_report(y_test_arg, y_pred_test))
```

```
              precision    recall  f1-score   support

           0       1.00      0.33      0.50         9
           1       0.38      0.18      0.24        17
           2       0.35      0.30      0.32        27
           3       0.37      0.69      0.48        26
           4       0.00      0.00      0.00         5

    accuracy                           0.38        84
   macro avg       0.42      0.30      0.31        84
weighted avg       0.41      0.38      0.35        84
```

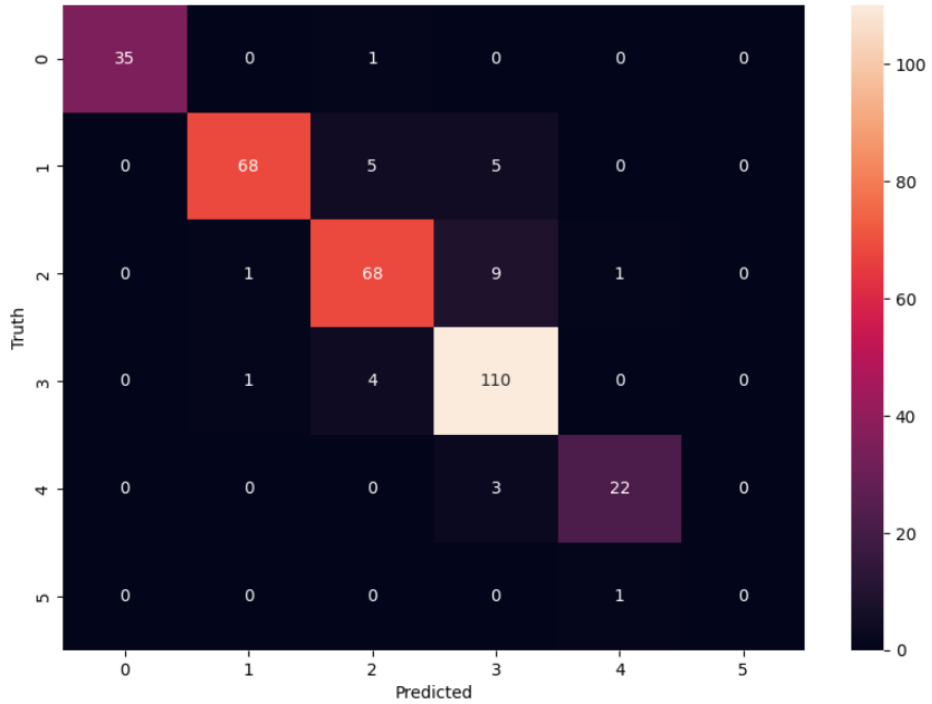
```
#Confustion Matrix for test data
cm=confusion_matrix(y_test_arg,y_pred_test)
plt.figure(figsize=(10,7))
sns.heatmap(cm,annot=True,fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.show()
```



```
#Classification Report for train data
print(classification_report(y_train_arg, y_pred_train))
```

```
              precision    recall  f1-score   support

           0       1.00      0.97      0.99        36
           1       0.97      0.87      0.92        78
           2       0.87      0.86      0.87        79
           3       0.87      0.96      0.91       115
           4       0.92      0.88      0.90        25
           5       0.00      0.00      0.00         1

    accuracy                           0.91       334
   macro avg       0.77      0.76      0.76       334
weighted avg       0.91      0.91      0.91       334
```

```
#Confustion Matrix for train data
cm1=confusion_matrix(y_train_arg,y_pred_train)
plt.figure(figsize=(10,7))
sns.heatmap(cm1,annot=True,fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.show()
```



Summary on simple Neural Network results:

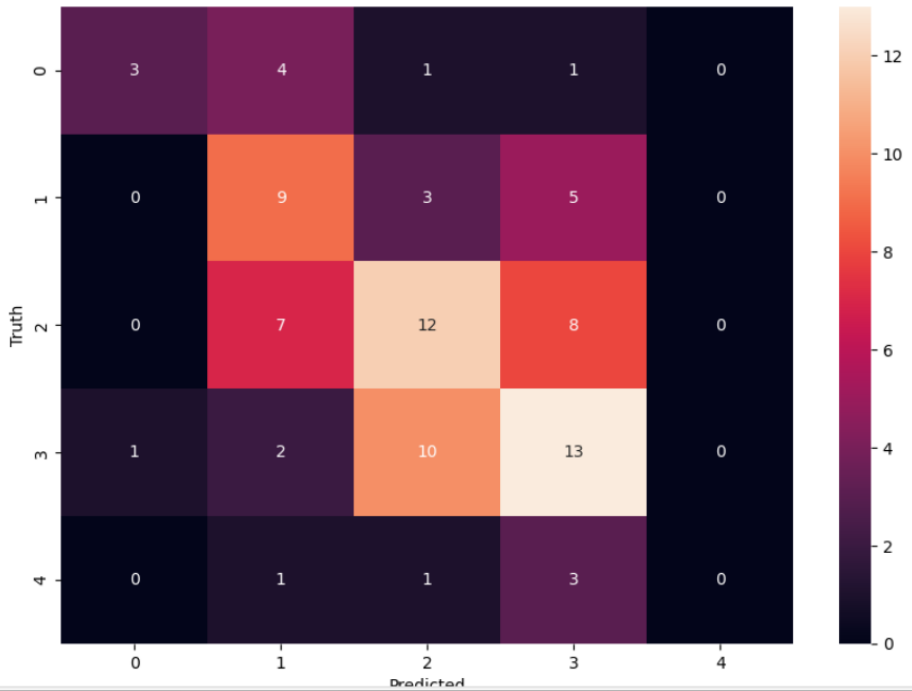
- Training data seems to be Overfitting.
- Performance matrix of Test data:
- Accuracy: 0.38
- Precision: 0.41
- Recall: 0.38
- F1-score: 0.35
- Accuracy curve results:
- Accuracy of training data drastically increased within few Epochs.
- There is no sudden increase or decrease in Validation Accuracy. Validation accuracy range is in between 40-50%.
- Loss curve results:
- Loss of training data decreased within few Epochs.
- Validation loss had reduced for initial few epochs but gradually increased.
- We could clearly see overfitting in training data.
- Validation data metrics are not too bad.

Tuned Neural Networks:

```
#Classification Report for test data
print(classification_report(y_test_arg_1, y_pred_test_1))
```

```
              precision    recall  f1-score   support

           0       0.75      0.33      0.46         9
           1       0.39      0.53      0.45        17
           2       0.44      0.44      0.44        27
           3       0.43      0.50      0.46        26
           4       0.00      0.00      0.00         5

    accuracy                           0.44        84
   macro avg       0.40      0.36      0.36        84
weighted avg       0.44      0.44      0.43        84
```

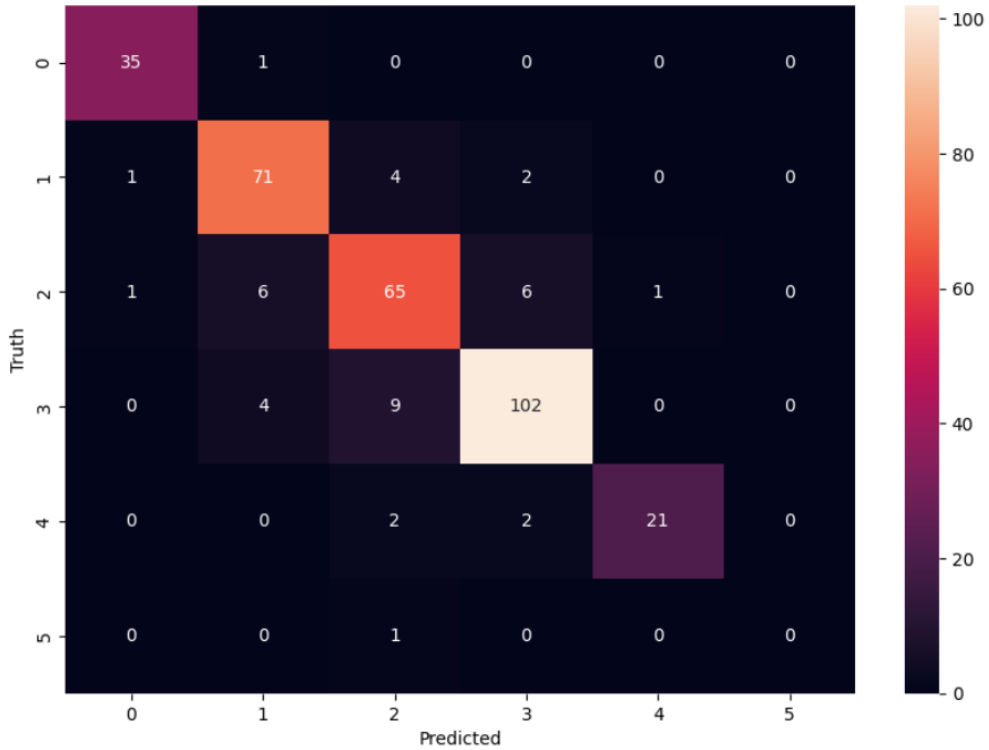
```python
#Confustion Matrix for test data
cm2=confusion_matrix(y_test_arg_1,y_pred_test_1)
plt.figure(figsize=(10,7))
sns.heatmap(cm2,annot=True,fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.show()
```




```python
#Classification Report for train data
print(classification_report(y_train_arg_1, y_pred_train_1))
```

```
              precision    recall  f1-score   support

           0       0.95      0.97      0.96        36
           1       0.87      0.91      0.89        78
           2       0.80      0.82      0.81        79
           3       0.91      0.89      0.90       115
           4       0.95      0.84      0.89        25
           5       0.00      0.00      0.00         1

    accuracy                           0.88       334
   macro avg       0.75      0.74      0.74       334
weighted avg       0.88      0.88      0.88       334
```

```
#Confustion Matrix for train data
cm3=confusion_matrix(y_train_arg_1,y_pred_train_1)
plt.figure(figsize=(10,7))
sns.heatmap(cm3,annot=True,fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.show()
```



Summary on Tuned Neural Network results:

- Traning data seems to be Overfitting . But better than previous NN model.
- Performance matrix of Test data:
- Accuracy : 0.44
- Precision : 0.44
- Recall : 0.44
- F1-score: 0.43
- Accuracy curve results:
- Accuracy of training and validation data is gradually increasing. Validation Accuracy had increased within few epochs but slightly decreased later.For most of the epochs , validation accuracy range is in between 40-50%.
- Loss curve results:
- Training loss curve looks good. There was a gradual decrease in Loss.

- Initally loss was high for Validation. But there was a sudden decrease in loss within few Epochs. And later it was gradually decreasing.

Conclusion on Neural Networks:

- We have built two Neural Network. One was simple neural network model and one tuned model. We have included Batch Normalization, Dropout and more hidden layers in tuned model.
- Overfitting in Training data was slightly reduced in Tuned model.
- Accuracy of tuned model was better.
- Accuracy and Loss curve of both Training data and testing data was better in tuned model.

Step 2: Design, train and test RNN or LSTM classifiers:

Here we will build three LSTM models (one simple LSTM model , one complex LSTM model and one LSTM model using SMOTE data) and then compare the result

Summary on simple LSTM model results:

- Performance matrix of Training data:
- Accuracy : 0.37
- Precision : 0.37
- Recall : 0.37
- F1-score: 0.23
- Performance matrix of Test data:
- Accuracy : 0.31
- Precision : 0.10
- Recall : 0.31
- F1-score: 0.15
- Accuracy curve results:
- Accuracy of training data was low initially but there was a sudden increase in Accuracy later. There is a slight fluctuation but training Accuracy curve looks ok after few epochs.
- There is no sudden increase or decrease in Validation Accuracy. Slightly decreased in initial few epochs but gradually increased later and was constant for last few epochs. Validation accuracy range is in between 30-35%.
- Loss curve results:
- Loss of training data was high initially but there was a sudden decrease in Loss later. Loss curve looks ok for last few epochs.
- Loss of validation data was high initially but there was a sudden decrease in Loss later. Loss curve looks good for last few epochs.
- Validation and training data metrics are ok.

Summary on complex LSTM model results:

- Performance matrix of Training data:
- Accuracy : 0.34
- Precision : 0.12
- Recall : 0.34
- F1-score: 0.18
- Performance matrix of Test data:
- Accuracy : 0.31
- Precision : 0.10
- Recall : 0.31
- F1-score: 0.15
- Accuracy curve results:
- There are slight fluctuations in Training data . Accuracy is increasing for few epochs and gradually decreasing for few.
- Accuracy curve of validation data is constant .
- Loss curve results:
- There is slight fluctuations in Training data. Loss is decreasing for few epochs and gradually increasing for few.
- Loss curve for validation data is low and constant.

Summary on LSTM model using SMOTE data results:

Performance matrix of Training data:

Accuracy : 0.59

Precision : 0.65

Recall : 0.59

F1-score: 0.57

Performance matrix of Test data:

Accuracy : 0.33

Precision : 0.36

Recall : 0.33

F1-score: 0.31

Accuracy curve results:

There are fluctuations in both validation and training data. Accuracy is increasing for few epochs and gradually decreasing for few.

Loss curve results:

There is slight fluctuation in training data. Loss is decreasing for few epochs and gradually increasing for few.

Loss curve for validation data is constant.

Conclusion on LSTM models:

- We have built three Neural Network. One was simple LSTM model , one was complex model and we use SMOTE balanced data for one model.
- We have included Batch Normalization, Max pooling , Dropout and more hidden layers in complex model.
- Techniques like tokenization, sequencing and Glove embedding performed on the source data
- We didn't notice overfitting on training data in any of these three models.
- Accuracy of LSTM model with balanced SMOTE data was better as compared to simple and complex LSTM model.
- There were fluctuations in Accuracy and Loss curves for all three models.
- Overall LSTM model with balanced SMOTE data gave better result.

Step 3: Choose the best performing classifier and pickle it

Here we have compared the results of each model and found the best performance classifier.

```
Comparision of Results
                        Method  Accuracy of Test data  Accuracy of Train data
0           Logistic Regression               0.440476                1.000000
1                 Decision Tree               0.345238                0.398204
2                 Random Forest               0.440476                1.000000
3                      Adaboost               0.345238                0.431138
4       Bagging (Decision Tree)               0.476190                0.994012
5       Bagging (NB Classifier)               0.416667                0.910180
6   Bagging (LogReg Classifier)               0.416667                1.000000
7                Gradient Boost               0.345238                1.000000
8                           SVM               0.321429                1.000000
9                            NN               0.380952                0.907186
10                     NN Tuned               0.440000                0.880000
11          LSTM (simple model)               0.309524                0.368263
12         LSTM (complex model)               0.309524                0.344311
13                   LSTM SMOTE               0.325926                0.589565
```

```
All performance matrics comparision of each models
```

	Method	Accuracy of Test data	Precision of Test data	Recall of Test data	F1-score of Test data	Accuracy of Train data	Precision of Train data	Recall of Train data	F1-score of Train data
0	Logistic Regression	0.440476	0.49	0.44	0.40	1.000000	1.00	1.00	1.00
1	Decision Tree	0.345238	0.21	0.35	0.20	0.398204	0.23	0.40	0.26
2	Random Forest	0.440476	0.47	0.44	0.39	1.000000	1.00	1.00	1.00
3	Adaboost	0.345238	0.35	0.35	0.23	0.431138	0.47	0.43	0.33
4	Bagging (Decision Tree)	0.476190	0.53	0.48	0.44	0.994012	0.99	0.99	0.99
5	Bagging (NB Classifier)	0.416667	0.41	0.42	0.36	0.910180	0.93	0.91	0.91
6	Bagging (LogReg Classifier)	0.416667	0.47	0.42	0.34	1.000000	1.00	1.00	1.00
7	Gradient Boost	0.345238	0.40	0.35	0.30	1.000000	1.00	1.00	1.00
8	SVM	0.321429	0.42	0.32	0.17	1.000000	1.00	1.00	1.00
9	NN	0.380952	0.41	0.38	0.35	0.907186	0.91	0.91	0.91
10	NN Tuned	0.440000	0.44	0.44	0.43	0.880000	0.88	0.88	0.88
11	LSTM (simple model)	0.309524	0.10	0.31	0.15	0.368263	0.37	0.37	0.23
12	LSTM (complex model)	0.309524	0.10	0.31	0.15	0.344311	0.12	0.34	0.18
13	LSTM SMOTE	0.325926	0.36	0.33	0.31	0.589565	0.65	0.59	0.57

Comparison of all performance metrics:

- We have added 9 ML models, 2 NN models and 3 LSTM models for comparison.
- Training data is overfitted in Logistic Regression, Random Forest, Bagging (Decision Tree) , Bagging (NB Classifier),Bagging (LogReg Classifier), Gradient Boost , SVM.
- So we can consider two ML models (Decision Tree, Adaboost) and 3 LSTM models for comparison.
- In ML models, Accuracy is 35% for testing data in both ML models. Accuracy of training data is slightly higher in Adaboost. Precision and Recall for both training and testing data is better in Adaboost. Precision and F1-score is higher for test data in AdaBoost as compared to Decision tree.
- Since the data is imbalanced, test data accuracy is same for both the models. we will compare the performance mertrics of each class. For class I both recall and f1 score is higher in Decision Tree but both models have same precision. For class II both the models have 0% performance metrics values. For class III, all the performance metrics is 0% in Decision Tree but we have 33% precision , 4% recall and 7% F1-score in AdaBoost. For class IV , recall and precision values are higher in Decision tree but f1-score is higher in AdaBoost. For class V , all the performance metrics is 0% in Decision Tree but we have 50% precision , 20% recall and 29% F1-score in AdaBoost. So in ML models we can consider Adaboost for comparison.
- In both the NN models training data is slightly overfitted. i.e. testing data have accuracy less than 50% but training data is greater than 88%. So we will not consider both the NN models for comparison.
- In LSTM models , Accuracy and precision and recall values are better in LSTM SMOTE. Accuracy and Loss curve looks better in LSTM SMOTE as compared to other two LSTM models .
- Since we have imbalanced data ,we will compare the performance mertrics of each class. We could see performance metrics for only CLASS III in LSTM Simple and Complex model. But since we have used the SMOTE to balance data , in LSTM SMOTE we could see the performance metrics for all the classes in LSTM SMOTE. Accuracy is 0.33 , Precision is 0.36 , Recall is 0.33 and F1-score is 0.31 in LSTM SMOTE.
- So in LSTM model we can consider LSTM SMOTE for comparison.
- Now to decide the final best model we will compare AdaBoost and LSTM SMOTE.
- Training data: All the performance metrics like Accuracy , Precision Recall and F1-score is higher in LSTM SMOTE as compared to AdaBoost.
- Test data: Accuracy and recall is slightly high in AdaBoost .Precision and F1-score is high in LSTM SMOTE.

- Since the data is imbalanced , we will compare the performance mertrics of each class. In LSTM SMOTE we have removed one of the class due to the imbalance and outlier. We didn't have any values for performance metrics in one of the class in AdaBoost. Precision is slighter higher in AdaBoost for all the four classes . Both Recall and F1-score is higher in LSTM SMOTE for three classes.

- Accuracy and Loss curve looks good for LSTM SMOTE.

- Training data accuracy is good in LSTM SMOTE. There are very small difference for Accuracy (2%) , Recall(2%) and precision (1%). But F1-score was 8% higher in LSTM SMOTE as compared to AdaBoost. Class wise performance metrics was better is LSTM SMOTE as compared to AdaBoost.

- AdaBoost uses a progressively learning boosting technique. Hence high-quality data is needed to get better result in AdaBoost . It is also very sensitive to outliers.

- Considering all this we can choose LSTM SMOTE as a best performing classifier.

Final Observations:

MILESTONE-1:

- We have first imported the data. Checked the info, shape, number of columns in the data.

- In the second step we have performed Data cleansing. Dropped duplicates. Check for unique values, NULL values. Drop unwanted columns. Visualise the missing values. Renamed few columns and check the outliers.

- In the third step we have performed Date preprocessing. Extract various columns like year, day, month and season. Convert description to lowercase. Remove special characters and stop words. Apply lemmatize and tokenizer on data.

- Additionally performed dummy variable encoding and label encoding. Checked correlation. Performed Univariate and Bivariate analysis of various columns and Visualized data.

- Fourth step is the Data Preparation. We have first removed columns which is not required for modelling. Checked for NULL values, duplicate records and standard deviation. Once we are satisfied with data, we have copied the data frame to csv and xlsx for future use.

- In the fifth step, we have first imported the pre-processed csv file. Description and Potential Accident level columns were used for all models. Split test and train data. We have then used bag of words feature extraction on the split data. Defined function to fit the model, calculate performance metrics and concatenate the accuracy results of each model. Built model codes which in turn calls functions to fit data, obtain accuracy and concatenate model results. We have re-used the functions and built nine Machine Learning models. Defined a Standard Scaler to normalize inputs. Set up the k-fold cross-validation. Used Pipeline on each model and used Grid search Hyper parameter tuning. Accuracy results, Classification Report results (Accuracy , Precision , Recall , F1-score) and Confusion Matrix results are displayed for each model. We have noticed training data overfitting in most of the models. On comparing the model results we have selected best model as AdaBoost.

- Additionally, we have exported the comparison results to csv for future use.

MILESTONE-2: Input is the Pre-processed output from Milestone-1.

- We have used existing train and test data (used by ML models).
- Performed few additional steps like reset and sort index , converting the categorical values to one-hot encoding , add missing column to test data, transform Labels into format acceptable by Neural Network.
- Designed, trained and test Neural networks classifiers. Here we built two Neural network models (one simple Neural network and one tuned neural network) and then compared the results. Applied Dense Neural Network with different activation functions and parameter tuning. We have included Batch Normalization , Dropout and more hidden layers in tuned model. Accuracy results , Classification Report results (Accuracy , Precision , Recall , F1-score), Confusion Matrix results , Accuracy and Loss curves are displayed for both Neural Network models. It yields good accuracy in training data but overfitting due to the complex model. Overfitting was slightly reduced in Tuned model. Accuracy of tuned model was better. Accuracy and Loss curve of both Training data and testing data was better in tuned model. We have appended the Neural Network models accuracy results with Machine Learning model results.
- Prior to building LSTM models , we have performed few techniques like tokenization , sequencing , define few parameters like max length , embedding size , max features and create a weight matrix using GloVe embeddings on source data.
- Designed, trained and test LSTM classifiers. Here we have built three LSTM models(one simple LSTM model , one complex LSTM model and one LSTM model using SMOTE data) and then compared the results. We have included Grid Search hyper parameter tuning in ML models. We have included Batch Normalization , Max pooling , Dropout and more hidden layers in complex model. Accuracy results , Classification Report results (Accuracy , Precision , Recall , F1-score), Confusion Matrix results , Accuracy and Loss curves are displayed for all the three LSTM models. We didn't notice overfitting on training data in any of these three models. Accuracy of LSTM model with balanced SMOTE data was better as compared to simple and complex LSTM model. There were fluctuations in Accuracy and Loss curves for all three models. But overall LSTM model with balanced SMOTE data gave better result. We have appended the LSTM models accuracy results with Machine Learning and Neural Network model results.
- Next step is to choose the best performing classifier and pickle it. On comparing nine Machine learning , two Neural Network and three LSTM models we have selected LSTM SMOTE as a best performing classifier and picked it.
- We have used potential accident level and description columns from pre-processed file and built the above models. We have selected one best model with better accuracy and pickled it. But accuracy was not very high for all these models.

- So we tried to get better accuracy using different column i.e. accident level. We have used accident level column along with description from existing pre-processed file, applied few additional pre-processing on these two columns and built one LSTM model. When we used the accident level column and built the LSTM model we had very good accuracy , Recall , Precision and F1-score for test data. It was 74%. There are slight fluctuations in Training loss curve but both Accuracy and Loss curve looks good. Hence we have pickled this model as well for future use.

CONCLUSION:

1. There were seven duplicates in the dataset, dropped them.
2. There were no missing values in the dataset.
3. Deleted unwanted features and created new features using Label encoding and Dummy variable/one hot encoding.
4. Applied NLP data pre-processing like converting to lower case. Removing special characters, Removing stop words, Lemmatization on Accident Decsription.
5. We have used Grid search for hyperparameter tuning in Machine Learning models.
6. Class imbalance issue is handled using SMOTE.
7. We were able to predict the potential accident level with a test accuracy of 48%.
8. We were able to predict the accident level with a test accuracy of 74%.
9. For potential accident level , by comparing the performance metrics of all nine Machine learning models , two Neural Network models and three LSTM models we have selected LSTM SMOTE as a best model and pickled it for future use.
10. For accident level , we have built only one LSTM model . Since this model had a good accuracy , precision , recall and f1-score we have selected this model and pickled it for future use.

LIMITATIONS:

1. We have less number of observations to analyse the cause of accidents correctly.
2. We have less number of features available in dataset.
3. The quality and relevance of the data can greatly impact the performance of the model.

IMPROVEMENTS:

1. Opting for Potential Accident Level as our target variables might have caused lower accuracy , precision and re-call when model was applied on raw data as well as over sampled data. We can use Accident Level as our target variable and compare the model results.
2. Adding more data. Large datasets can provide more accurate and robust models.
3. Machine learning (ML) models generally consume a lot of training data due to the large number of tunable parameters and lack of priors. So we can either use simpler models or build in priors to reduce data requirements during learning and hence making the models learn faster and more efficiently with possibility of better generalization performance.
4. We have to explore and implement more preprocessing techniques.
5. We have to explore and implement more feature engineering and feature selection techniques.
6. Increase model capacity. To increase the capacity, we add layers and nodes to a deep network gradually.
7. Model & dataset design changes. Analyse bad predictions in validation data, Shuffle the dataset. Balance the dataset so that each class has similar amount of samples. Monitor the activations. Monitor the percentage of dead nodes. If the deep network has a huge amount of dead nodes, we should trace the problem further.
8. We have to dig deeper into performance metrics. Performance metrics can be improved by using PCA or by increasing dimensions or by using learning rate tuning, dropouts, early stopping and hyper parameter tuning more efficiently.
9. Overfitting can be reduced by stopping the training when the validation errors increase persistently.